CS 350 Project Specifications, Version 0.1

Description

This living document specifies how to build the communication interface that interprets English-like text commands from the end user to configure and manipulate the rest of the system we have presumably built for the customer. The goal is to parse and process the commands and hand them off to the architecture for execution.

Part 1 is to build the command line interface that allows a user to create, connect, and manipulate a representative subset of the components we have investigated throughout the quarter.

Part 2 (to be released later) demonstrates through formal testing how well the complete solution works. It does not correspond exactly to validation because we lack a good description of the customer's needs, but it does play the role of system testing.

These specifications serve to guide you both as a programmer in Part 1 and a tester and notional end user in Part 2. All specifications must be satisfied, but in some cases, the architectural already does so. In other words, a constraint may apply to the end user, not to the programmer. Use your best judgment and ask for clarification. Use your experience from Task 4 here.

PART 1

Definitions

The commands reference the following grammatical fields, the type of each being a class in `sbw.architecture.datatype`.

Field	Type	Definition
`acceleration`	`Acceleration`	A nonnegative real in appropriate units per clock tick
`angle`	`Angle`	A nonnegative real in degrees
`id`	`Identifier`	An arbitrary alphanumeric identifier, like Java variables
`percent`	`Percent`	A real in percent [0,100]
`position`	`Position`	A closed set of flap positions [*up*,1,2,3,4]
`power`	`Power`	A real in percent [0,100]
`rate`	`Rate`	A positive integer clock rate in milliseconds
`speed`	`Speed`	A positive real in appropriate units per clock tick

Real values must have an initial digit before the decimal; *e.g.*, `0.1`, not `.1`.

Commands

Carets, parentheses, and square brackets are not part of commands. Vertical bar indicates logical *or*; asterisk indicates zero or more instances of the preceding term or parenthetical group; plus indicates one or more. Square brackets indicate an optional group.

White space, except in literals, does not matter. All text except identifiers is case insensitive.

Commands may appear on the same line if they are separated by a semicolon. A line is always terminated by a newline.

A comment prefixed with `//` may follow a command or be on its own line.

All identifiers must be unique.

Issuing a command terminates any pending requests within the affected components.

All failure modes must be accounted for with appropriate error handling. The messages and delivery mechanism need not be elaborate or particularly user-friendly.

Use only standard Java (8 or higher), no external tools, libraries, etc.

Teams of two skip commands with an asterisk (TBD).

Architecture

Download `project-bundle-v1_0.zip`, which contains `project-jar-v1_0.jar` and `CommandLineInterface.java`.

To connect to the JAR in Eclipse, right click on the project and select Build Path→Configure Build Path→Libraries→Add External JARs.

The provided skeleton class for the command-line interface is called `sbw.project.cli.CommandLineInterface`. Rename your parser `sbw.project.cli.parser.CommandParser` and supply it with `_actionSet` and `command` as indicated in `processInput()`. Execute the system from `main()` in `sbw.project.RunProject`. It will present this interface, where `:` means output and `>` means input. Enter your commands here.

```
: Welcome to your fly-by-wire system!
>
```

Keyboard input should appear in green. Note that the console input behavior in Eclipse may be a little strange, sometimes putting the cursor in the wrong spot, even though it is still interpreted correctly.

All `Actuator`*X* classes are below package `sbw.project.components.actuator`, and `Controller`*X* classes are in `sbw.project.components.controller`. All `do`*X* methods are defined below. See the Javadoc for the method signatures.

I. Creational Commands

Creational commands allow the user to define and declare actuators.

All `do`*X* methods in this section are in class `sbw.project.cli.action.ActionCreational`.

Each of these commands instantiates the appropriate `Actuator`*X* class with the given arguments (and verifies their validity), registers it with the `MapActuator`, and writes this event to the command-line interface via `sbw.project.cli.action.A_Action.processOutput()`.

1. Rudder

a. `CREATE RUDDER <id> WITH LIMIT <angle> SPEED <speed> ACCELERATION <acceleration>`

Creates an `ActuatorRudder` with identifier `id` that deflects `angle` degrees left (negative) or right (positive) from neutral (0 degrees) at maximum speed `speed` and acceleration `acceleration`.

This calls `doCreateRudder()`, which creates and registers an instance of `ActuatorRudder`.

2. Elevator

a. `CREATE ELEVATOR <id> WITH LIMIT <angle> SPEED <speed> ACCELERATION <acceleration>`

Creates an `ActuatorElevator` with identifier `id` that deflects `angle` degrees up (positive) or down (negative) from neutral (0 degrees) at maximum speed `speed` and acceleration `acceleration`.

This calls `doCreateElevator()`, which creates and registers an instance of `ActuatorElevator`.

3. Aileron

a. `CREATE AILERON <id> WITH LIMIT UP <angle1> DOWN <angle2> SPEED <speed> ACCELERATION <acceleration>`

Creates an `ActuatorAileron` with identifier `id` that deflects `angle1` degrees up (positive) from neutral (0 degrees) and `angle2` degrees down (negative) at maximum speed `speed` and acceleration `acceleration`.

This calls `doCreateAileron()`, which creates and registers an instance of `ActuatorAileron`.

4. Flap

a. `CREATE SPLIT FLAP <id> WITH LIMIT <angle> SPEED <speed> ACCELERATION <acceleration>`

Creates an `ActuatorFlapSplit` with identifier `id` that deflects `angle` degrees down from center (0 degrees) at maximum speed `speed` and acceleration `acceleration`.

This calls `doCreateFlap()`, which creates and registers an instance of `ActuatorFlapSplit`.

b. `CREATE FOWLER FLAP <id> WITH LIMIT <angle> SPEED <speed> ACCELERATION <acceleration>`

Creates an `ActuatorFlapFowler` with identifier `id` that deflects `angle` degrees down from center (0 degrees) at maximum speed `speed` and acceleration `acceleration`. Additional hardcoded definitions are specified in `ActuatorFlapFowler`.

This calls `doCreateFlap()`, which creates and registers an instance of `ActuatorFlapFowler`.

5. Engine

a. `CREATE ENGINE <id> WITH SPEED <speed> ACCELERATION <acceleration>`

Creates an `ActuatorEngine` with identifier `id` with maximum speed `speed` and acceleration `acceleration`. The interval is always [0..100] percent power.

This calls `doCreateEngine()`, which creates and registers an instance of `ActuatorEngine`.

6. Gear

a. `CREATE NOSE GEAR <id> WITH SPEED <speed> ACCELERATION <acceleration>`

Creates an `ActuatorGearNose` with identifier `id` with maximum speed `speed` and acceleration `acceleration`. The interval is always [0..100] percent extended.

This calls `doCreateGearNose()`, which creates and registers an instance of `ActuatorGearNose`.

b. `CREATE MAIN GEAR <id> WITH SPEED <speed> ACCELERATION <acceleration>`

Creates an `ActuatorGearMain` with identifier `id` with maximum speed `speed` and acceleration `acceleration`. The interval is always [0..100] percent extended.

This calls `doCreateGearMain()`, which creates and registers an instance of `ActuatorGearMain`.

II. STRUCTURAL COMMANDS

Structural commands connect controllers to buses and the command-line interface.

All `do`*X* methods in this section are in class `sbw.project.cli.action.ActionStructural`.

Each of these commands instantiates the appropriate `Controller`*X* class with the given arguments (and verifies their validity), registers it with the `MapController`, and writes this event to the command-line interface via `sbw.project.cli.action.A_Action.processOutput()`.

1. Rudder

a. `DECLARE RUDDER CONTROLLER <id1> WITH RUDDER <id2>`

Creates a `ControllerRudder` with identifier `id1` containing rudder `id2`.

This calls `doDeclareRudderController()`, which creates and registers an instance of `ControllerRudder`.

2. Elevator

a. `DECLARE ELEVATOR CONTROLLER <id1> WITH ELEVATORS <id2> <id3>`

Creates a `ControllerElevator` with identifier `id1` containing elevators `id2` (left) and `id3` (right), which must be identical in configuration.

This calls `doDeclareElevatorController()`, which creates and registers an instance of `ControllerElevator`.

3. Aileron

a. `DECLARE AILERON CONTROLLER <id1> WITH AILERONS <id`*n*`>+ PRIMARY <id`*x*`>`
`(SLAVE <idslave> TO <idmaster> BY <percent> PERCENT)*`

Creates a `ControllerAileron` with identifier `id1` containing *n* ailerons `id`*n*, where *n* is even. The first half of *n* in order are on the left wing, and the second half on the right. `id`*x* specifies which of `id`*n* is the primary one that is directly commanded by a request to this controller. It must be on the left wing. The others respond based on it: all ailerons on the same side deflect in the same direction, and all on the opposite side deflect in the opposite direction (except when used as a speed brake; see III.3.b). `idslave` optionally bases its deflection (and likewise its opposite's) on `idmaster` by `percent` percent. Any chain of mixing is possible, but there are no cyclical relationships. Aileron configurations must be symmetrically identical.

This calls two variants of `doDeclareAileronController()`, which creates and registers an instance of `ControllerAileron`.

4. Flap

a. `DECLARE FLAP CONTROLLER <id> WITH FLAPS <id`*n*`>+`

Creates a `ControllerFlap` with identifier `id` containing flaps `id`*n*, where *n* must be even and the flap types must have symmetrically identical configurations.

This calls `doDeclareFlapController()`, which creates and registers an instance of `ControllerFlap`.

5. Engine

a. `DECLARE ENGINE CONTROLLER <id1> WITH ENGINE[S] <id`*n*`>+`

Creates a `ControllerEngine` with identifier `id1` containing engines `id`*n*, arrayed left to right in order, with identical configurations. The plural form of `ENGINE` need not correspond grammatically to *n*.

This calls `doDeclareEngineController()`, which creates and registers an instance of `ControllerEngine`.

6. Gear

a. `DECLARE GEAR CONTROLLER <id1> WITH GEAR NOSE <id2> MAIN <id3> <id4>`

Creates a `ControllerGear` with identifier `id1` containing nose gear `id2` and main gear `id3` (left) and `id4` (right). Both main gear must be identical in configuration.

This calls `doDeclareGearController()`, which creates and registers an instance of `ControllerGear`.

7. Bus

a. `DECLARE BUS <id1> WITH CONTROLLER[S] <id`*n*`>+`

Creates a `Bus` with identifier `id1` containing controllers `id`*n* and registers it with `CommandLineInterface`. The plural form of `CONTROLLER` need not correspond grammatically to *n*.

This calls `doDeclareBus()`.

8. Commit

a. `COMMIT`

Finalizes the creational and structural stages of building the network. No commands in Section I or II are valid after this point. Only Section III commands are, and only after this point.

This calls `doCommit()`.

III. BEHAVIORAL COMMANDS

Behavioral commands allow the user to manipulate the connected components from Sections I and II. They are not available until a commit command (II.8.a) is issued.

All `doX` methods in this section are in class `sbw.project.cli.action.ActionBehavioral`, and the commands are in `sbw.project.cli.action.command.behavioral`.

1. Rudder

a. `DO <id> DEFLECT RUDDER <angle> LEFT|RIGHT`

Requests that rudder controller `id` deflect its rudder respectively left or right to angle `angle`.

This calls `submitCommand()` with an instance of `CommandDoDeflectRudder`.

2. Elevator

a. `DO <id> DEFLECT ELEVATOR <angle> UP|DOWN`

Requests that elevator controller `id` deflect its elevators respectively up or down to angle `angle`.

This calls `submitCommand()` with an instance of `CommandDoDeflectElevator`.

3. Aileron

a. `DO <id> DEFLECT AILERONS <angle> UP|DOWN`

Requests that aileron controller `id` deflect its primary aileron respectively up or down to angle `angle`. The opposite ailerons will deflect in the other direction. The slave ailerons will derive their angles as specified in the slave clause in II.3.a, if one was given; otherwise, the relationship is 100%.

This calls `submitCommand()` with an instance of `CommandDoDeflectAilerons`.

b. `DO <id> SPEED BRAKE ON|OFF`

Requests that aileron controller `id` deflect all its ailerons respectively to maximum up or neutral (0 degrees) to serve as speed brakes. Slave mixing does not apply.

This calls `submitCommand()` with an instance of `CommandDoDeploySpeedBrake`.

4. Flap

a. `DO <id> DEFLECT FLAP <position>`

Requests that flap controller `id` deflect its flaps to position `position`, which is correspondingly 0, 25, 50, 75, or 100% of the deflection range.

This calls `submitCommand()` with an instance of `CommandDoSetFlaps`.

5. Engine

a. `DO <id> SET POWER <power>`

Requests that engine controller `id` set the power of all engines to `power`.

This calls `submitCommand()` with an instance of `CommandDoSetEnginePowerAll`.

b. `DO <id1> SET POWER <power> ENGINE <id2>`

Requests that engine controller `id1` set the power of engine `id2` to `power`.

This calls `submitCommand()` with an instance of `CommandDoSetEnginePowerSingle`.

6. Gear

a. `DO <id> GEAR UP|DOWN`

Requests that gear controller `id` respectively raise or lower its gear.

This calls `submitCommand()` with an instance of `CommandDoSelectGear`.

7. General

a. `HALT <id>`

Requests that controller `id` terminate the current process for itself and its actuators.

This calls `submitCommand()` with an instance of `CommandDoHalt`.

IV. MISCELLANEOUS COMMANDS

All `do`*X* methods in this section are in class `sbw.project.cli.ActionMiscellaneous`, and the commands are in `sbw.project.cli.action.command.misc`.

a. `@CLOCK <rate>`

Sets the system clock speed to `rate` ticks per second.

This calls `submitCommand()` with an instance of `CommandDoSetClockRate`.

b. `@CLOCK PAUSE|RESUME|UPDATE`

Instructs the communication system to pause or resume the system clock, respectively, or to force it to advance a tick. Force is valid only when the clock is paused.

This calls `submitCommand()` with an instance of `CommandDoSetClockRunning` or `CommandDoClockUpdate`.

c. `@CLOCK`

Outputs the clock rate to the command-line interface as "`clock = <rate>`", or "`clock = paused`" if it is not running.

This calls `submitCommand()` with an instance of `CommandDoShowClock`.

d. `@RUN "<filename>"`

Loads a text file with commands of the form here, one per line, and executes them in order. `filename` is any valid filename with path and extension. The quotes are part of the command. Output each command to the command-line interface via `sbw.project.cli.CommandLineInterface.processOutput()`.

This calls `submitCommand()` with an instance of `CommandDoRunCommandFile`.

e. `@EXIT`

Exits the system.

This calls `submitCommand()` with an instance of `CommandDoExit`.

f. `@WAIT <rate>`

Waits `rate` ticks before executing the next behavioral command. This command is not valid until after `@commit`.

This calls `submitCommand()` with an instance of `CommandDoWait`.